

Managing Packages and Repos

1: CONFIGURING A YUM REPOSITORY

To enable support for a third-party repository on a managed host, Ansible provides the `yum_repository` module.

Declaring a Yum Repository

1: The ``yum_repository`` module creates or updates a ``.repo`` file under:

...

`/etc/yum.repos.d/<file>.repo`

...

Example:

```
``yaml
```

```
yum_repository:
```

```
  file: example
```

```
  name: example-internal
```

```
  description: Example Inc. Internal YUM repo
```

```
  baseurl: http://materials.example.com/yum/repository/
```

```
  enabled: yes
```

```
  gpgcheck: yes
```

```
...
```

This produces:

...

`/etc/yum.repos.d/example.repo`

...

Containing:

...

```
[example-internal]
```

```
baseurl = http://materials.example.com/yum/repository/
```

```
enabled = 1
```

```
gpgcheck = 1
```

```
name = Example Inc. Internal YUM repo
```

...

Key parameters

- **file:** name of the `.repo` file (Ansible adds `.repo`)
- **name:** repo ID inside the file
- **description:** human-readable name
- **baseurl:** URL or file path (`file:///myrepo/BaseOS`)
- **enabled:** 1/0
- **gpgcheck:** enforces GPG signature validation
- **state:**
 - `present` → create/update
 - `absent` → delete the repo file



✓ 2. You Must Use `rpm_key` When `gpgcheck=yes`

When `gpgcheck: yes` is enabled, `Yum` refuses to install packages unless the `**public GPG key**` is installed.

Example:

```
``yaml
```

```
rpm_key:
```

```
  key: http://materials.example.com/yum/repository/RPM-GPG-KEY-example
```

```
  state: present
```

```
...
```

This imports the key into the RPM database:

```
...
```

```
rpm -qa gpg-pubkey*
```

```
...
```

Without this, Yum will fail with:

```
...
```

Public key for `<package>.rpm` is not installed

```
...
```

✓ 3. Correct Playbook Structure

Recommended order

1. Import GPG key
2. Create repository
3. Install packages

```
``yaml
```

```
---
```

```
- name: Configure internal Yum repository
```

```
  hosts: servera.lab.example.com
```

```
  become: yes
```

```
  tasks:
```

```
    - name: Install GPG key
```

```
      rpm_key:
```

```
        key: http://materials.example.com/yum/repository/RPM-GPG-KEY-example
```

```
        state: present
```

```
    - name: Configure internal repository
```

```
      yum_repository:
```

```
        file: example
```

```
        name: example-internal
```

```
        description: Example Inc. Internal YUM repo
```

```
        baseurl: http://materials.example.com/yum/repository/
```

```
        enabled: yes
```

```
        gpgcheck: yes
```

```
        state: present
```

```
...
```

✓ 4. When NOT to use `yum\_repository`

Some third-party repos (like **EPEL**) provide a `.repo` file inside an RPM package.

Example:

```
...
```

```
epel-release-latest-8.noarch.rpm
```

```
...
```

In such cases, you should install the RPM instead:

```
``yaml
```

```
yum:
```

```
  name: https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm
```

```
  state: present
```

```
...
```

This automatically:

- installs `/etc/yum.repos.d/epel.repo`

- installs the GPG key

✓ 5. Advanced Notes (Important for Real Environments)

a. Local repositories (`file://`)

```
...
```

```
/myrepo/AppStream
```

```
/myrepo/BaseOS
```

```
...
```

Your repo definitions should be:

```
``yaml
```

```
baseurl: file:///myrepo/AppStream
```

```
...
```

b. Metadata cleanup

When removing a repo:

```
...
```

```
state: absent
```

```
...
```

The `.repo` file is deleted, but metadata remains until:

```
...
```

```
yum clean all
```

```
...
```

Use a handler:

```
``yaml
```

```
notify: clean yum metadata
```

```
...
```

PlayBook:

```
...
```

```
- name: Remove old repository and clean metadata
```

```
  hosts: all
```

```
  become: yes
```

```
  tasks:
```

```
    - name: Remove old internal repository
```

```
      yum_repository:
```

```
        name: example-internal
```

```
        file: example
```

```
        state: absent
```

```
      notify: clean yum metadata
```

```
  handlers:
```

```
    - name: clean yum metadata
```

```
      command: dnf clean all
```

```
...
```

2:Create Repo using Ansible Role:

✓ 1. Directory Structure

...

project/

└─ playbook.yml

└─ roles/

└─ local_repo/

└─ tasks/

└─ main.yml

└─ templates/

└─ local.repo.j2

└─ defaults/

└─ main.yml

└─ handlers/

└─ main.yml

...

✓ 2. defaults/main.yml*

(Variables the user can override)

```
``yaml
```

```
repo_dir: /myrepo
```

```
repo_file: local
```

```
repo_id: localrepo
```

```
repo_name: Local Repository
```

```
repo_baseurl: "file:///myrepo"
```

```
repo_enabled: 1
```

```
repo_gpgcheck: 0``
```

...

✓ 3. templates/local.repo.j2

(Uses your `ansible\_managed` header)

```
````jinja2
{{ ansible_managed }}

[{{ repo_id }}]
name={{ repo_name }}
baseurl={{ repo_baseurl }}
enabled={{ repo_enabled }}
gpgcheck={{ repo_gpgcheck }}
...

```

### ✓ 4. tasks/main.yml

```
````yaml
- name: Ensure local repo directory exists
  file:
    path: "{{ repo_dir }}"
    state: directory

- name: Create repository metadata
  command: createrepo {{ repo_dir }}
  notify: clean yum metadata

- name: Deploy local repo configuration from template
  template:
    src: local.repo.j2
    dest: "/etc/yum.repos.d/{{ repo_file }}.repo"
  notify: clean yum metadata
...

```

✓ 5. handlers/main.yml

```
``yaml
```

```
- name: clean yum metadata
```

```
  command: "{{ ansible_pkg_mgr }}" clean all"
```

```
``
```

This automatically uses:

```
- `dnf` on RHEL 8/9
```

```
- `yum` on RHEL 7
```

✓ 6. Top-Level Playbook (playbook.yml)

```
``yaml
```

```
- name: Configure local repository
```

```
  hosts: all
```

```
  become: yes
```

```
  roles:
```

```
    - local_repo
```

```
``
```


MANAGING PACKAGES WITH ANSIBLE

✓ Managing Packages with Ansible

Ansible manages software packages on RHEL-based systems using the **yum** module (or **dnf** on newer systems). The module interacts with the system's package manager to install, remove, or update packages.

1. Basic yum module usage

Install a package

- name: Install httpd

yum:

name: httpd

state: present

Equivalent to:

yum install httpd

Remove a package

- name: Remove httpd

yum:

name: httpd

state: absent

Equivalent to:

yum remove httpd

Install or update to latest version

- name: Install or update httpd

yum:

name: httpd

state: latest

Equivalent to:

yum update httpd

2. Installing multiple packages efficiently

Best method — list under name

...

- name: Install multiple packages

yum:

name:

- httpd

- mod_ssl

- httpd-tools

state: present

...

✓ Single Yum transaction

✓ Fastest

✓ Recommended

Equivalent to:

yum install httpd mod_ssl httpd-tools

✗ Avoid using loops

- name: Install packages (slow)

yum:

name: "{{ item }}"

state: present

loop:

- httpd

- mod_ssl

- httpd-tools

This performs **three separates yum transactions**, which is slower.

3. Installing package groups

Yum groups must be prefixed with @.

Install Development Tools

- name: Install Development Tools

yum:

name: "@Development Tools"

state: present

Equivalent to:

yum group install "Development Tools"

Remove group

- name: Remove Development Tools

yum:

name: "@Development Tools"

state: absent

4. Installing AppStream modules

Modules also use @ prefix.

Example: Install Perl 5.26 minimal profile

- name: Install perl module

yum:

name: "@perl:5.26/minimal"

state: present

Equivalent to:

yum module install perl:5.26/minimal

With the yum Ansible module, you must prefix group names with @. Remember that you can retrieve the list of groups with the yum group list command.

5. Gathering package facts

Use `package_facts` to collect installed package information.

The `package_facts` Ansible module collects the installed package details on managed hosts. It sets the `ansible_facts.packages` variable with the package details.

Playbook:

```
...

- name: Gather installed packages

  hosts: servera.lab.example.com

  tasks:

    - name: Collect package facts

      package_facts:

        manager: auto

    - name: Show all installed packages

      debug:

        var: ansible_facts.packages

    - name: Show NetworkManager version

      debug:

        msg: "Version {{ ansible_facts.packages['NetworkManager'][0].version }}"

        when: "'NetworkManager' in ansible_facts.packages"

...
```

This gives you a dictionary of all installed packages.

`manager: auto` tells Ansible:

"Detect the correct package manager automatically."

6. Using the generic package module

The `yum` Ansible module works on managed hosts that are using the Yum Package Manager.

For other package managers, Ansible usually provides a dedicated module. For example, the `dnf` module manages packages on operating systems such as Fedora using the DNF package manager. The `apt` module uses the APT package tool available on Debian or Ubuntu. The `win_package` module can install software on Microsoft Windows systems

The **package** module automatically selects the correct backend:

- `yum` on RHEL
- `dnf` on Fedora
- `apt` on Ubuntu

As an alternative, the generic `package` module automatically detects and uses the package manager available on the managed hosts

Example:

```
...
```

```
- name: Install httpd
```

```
  package:
```

```
    name: httpd
```

```
    state: present
```

```
...
```

⚠ BUT:

Package names differ across OSes (e.g., `httpd` vs `apache2`), so conditionals may still be required.

7. Conditional package installation

Example for mixed RHEL + Fedora environment:

...

- name: Install httpd on RHEL

yum:

name: httpd

state: present

when: ansible_distribution == "RedHat"

- name: Install httpd on Fedora

dnf:

name: httpd

state: present

when: ansible_distribution == "Fedora"

...

Tips

- Use **lists**, not loops, for multiple packages.
 - Use **@groupname** for group installs.
 - Use **package_facts** to inspect installed software.
 - Use **state: latest** for updates.
 - Use **package** module only when you don't need advanced yum/dnf features.
 - Know how to install AppStream modules (@module:stream/profile).
-